Using Epistemic Observability in Agentic Systems for Combating Hallucinations

Tony Mason
The University of British Columbia

Vaastav Anand
Max Planck Institute for Software Systems

Abstract

LLMs are increasingly deployed as components of production systems, introducing a fundamental challenge: generated outputs may contain fabrications, while verification incurs computational and monetary cost. Consequently, systems cannot afford to verify every generation equally and must instead decide how to allocate a limited verification budget. This raises a fundamental question: what reliability does each level of verification investment buy?

We argue that text-only observation, even with bounded supervision, is structurally insufficient for reliably assessing the epistemic validity of model outputs, regardless of model scale or training procedure. To address this limitation, we introduce epistemic observability, an interface that exposes internal computational telemetry alongside generated text; these signals provide systems with richer evidence for estimating hallucination risk and prioritizing verification effort. We present a four-tier epistemic observability hierarchy that characterizes the trade-off between verification cost and the amount of epistemic information available to downstream systems. As a first realization of this framework, we design a tensor-generation interface that exposes lightweight epistemic telemetry and demonstrate that tensor-guided judges consistently outperform text-only baselines across all verification budgets, improving accuracy by 3.2–4.7 percentage points on average across four model families.

1 Introduction

Systems incorporating language model components face a verification problem. The components generate text by sampling from learned probability distributions, and some fraction of that text is fabricated: fluent, confident, and wrong. We use the term *fabrication* throughout this paper to emphasize that the system's default behavior under coherence optimization is reasonable and not a malfunction to be patched. Xu et al. [20] prove that fabrication is inevitable for LLMs over the class of computable functions.

Consequently, every system that deploys a language model component must allocate a budget: how much of the output to verify, which outputs to prioritize, and what signals to use for triage. Verification cost varies by mechanism: a citation lookup costs latency, a second model as judge costs compute, checking a summary against its source costs more still.

The fundamental source of these costs is the interface through which language models are observed. Today's systems expose only the generated text while hiding the model's internal computation, which contains signals that may distinguish grounded generation from fabrication. Because these signals are discarded before reaching the supervisor, they cannot be recovered from the output alone. In an accompanying technical report [12], we show that, under bounded supervision, grounded and fabricated internal states can be observationally equivalent through the text channel alone, independent of model scale or training procedure.

This observability gap has direct consequences for verification cost: a supervisor limited to text spends its budget reconstructing information that was available during generation but hidden by the interface, instead of focusing on the small fraction of outputs that genuinely require scrutiny. This paper therefore asks the systems question: if language models exposed richer observational interfaces, how would that change the cost, effectiveness, and return on investment of verification?

Our key insight is that these hidden signals need not remain hidden. During inference, language models naturally produce auxiliary information that reflects their internal computation. We refer to these signals as *telemetry*: byproducts of inference that the model does not explicitly choose to communicate. This distinguishes telemetry from the model's *testimony*—the generated text itself. Whereas testimony is optimized to satisfy the prompting objective and can therefore be misleading, telemetry arises as a consequence of computation: under current training objectives the model cannot alter it without altering the computation itself. This is an empirical property of today's training pipelines, not an architectural guarantee, and we treat it as such (Figure 3.2).

We introduce *epistemic observability*: an interface that exposes this telemetry alongside the standard text output. Unlike interpretability, which explains why a model activates particular neurons, or explainability, which generates post-hoc rationales, epistemic observability surfaces byproducts of inference that help estimate whether the generated content is grounded in the model's knowledge or likely fabricated.

To support a range of applications, we propose a multi-tier epistemic observability interface, where each successive tier exposes richer telemetry about the generation process at greater computational cost. A creative brainstorming assistant may require only Tier 0 observability, whereas a medical decision-support system may justify the cost of higher tiers;

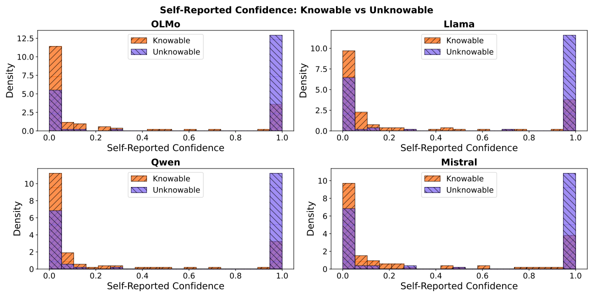

Figure 1. Self-reported confidence is inverted: all four models report higher confidence on unknowable queries

the hierarchy makes explicit what additional epistemic information each level of investment provides.

As a proof of concept, we build a tensor-generation interface exposing per-token entropy (Tier 1) and attention summaries (a low-cost Tier 2 signal), and show that tensor-guided judges outperform text-only baselines at every verification budget, by 3.2–4.7 percentage points on average across four model families.

2 Background & Motivation

A language model maps input tokens to output tokens, producing internal signals (entropy, attention patterns, logprobabilities) as computational byproducts. The training corpus contains both truth-tracking institutional text and misinformation. Probing how a model trained on this mixture responds yields four canonical query types that span the verification space: *adversarial truths* (true but implausible), *shattered lies* (no training-data grounding), *deceived lies* (fabrications that pattern-match plausible language), and *confused truths* (counterintuitive facts).

A model trained on this corpus inherits its mixed regularities: base models fabricate substantially on factual benchmarks, yet also encode internal representations of correctness that can be elicited under the right conditions [3]. The model is neither reliably honest nor uniformly deceptive; the question for systems engineers is not whether it is honest, but what it costs to verify when it is not.

2.1 Testimony & Telemetry

Architecture primer. A transformer generates text one *token* (subword unit) at a time: at each step it produces a probability distribution over its vocabulary, selects a token, and conditions the next step on all prior output. Each step passes through a stack of layers producing *attention patterns* (weighted relevance scores between positions) and intermediate representations. The *entropy* of the output distribution, the attention patterns, and the *log-probabilities* of selected tokens are byproducts of this computation.

Existing models expose only text. The text is the testimony; the telemetry are measurements of the computation that produced it. From a cost perspective, testimony is cheap to produce and inspect but unreliable; telemetry is more expensive to export and process but, under current training regimes, harder to game, because it is a byproduct of the computation rather than a product of the optimization objective (Figure 3.2).

2.2 The Text Channel Cannot Self-Verify

Current models cannot be trusted to honestly report their own epistemic state through the text channel. To demonstrate the inadequacy of text-only verification, we use a taxonomy of query types defined by construction rather than by inspecting any model's training data: *knowable* queries have a determinate, publicly verifiable answer (e.g., national capitals, well-documented scientific facts), whereas *unknowable* queries have no correct answer for any model, because the entity does not exist, the event has not happened, or the information is private. "Knowable" is a property of the query, not a claim about a particular model's weights; a model may still fail a knowable query, and the evaluator scores that failure. We report each model's self-confidence score for the self-reported confidence baseline, the cheapest verification strategy available to a bounded supervisor. In this baseline, the supervisor asks the model "How confident are you?" after generation and tests whether the model can introspect and report its epistemic state through text.

Figure 1 shows that self-reported confidence is *inverted*: all four models report higher confidence on unknowable queries than on knowable ones, yielding AUC 0.28–0.36 across architectures (all below random). A user who asks "how confident are you?" receives a number that anticorrelates with epistemic grounding, and an adversary could exploit the inversion to make fabrications appear maximally confident. Under bounded oversight, a text-only supervisor cannot distinguish truthful uncertainty from confident fabrication.

3 Epistemic Observability

We define epistemic observability as the ability of a language model to expose telemetry about its epistemic state during inference: signals produced as byproducts of generation—decoding statistics, internal representations, computational traces—that give downstream systems evidence about whether a response is likely to be fabricated.

3.1 Observability Tiers

We present a four-tier epistemic observability hierarchy that characterizes the trade-off between verification cost and the amount of epistemic information available to downstream systems for assessing reliability of the generated text.

Tier 0: Output Observability. Tier 0 exposes only the generated text, treating the model as a black box. This is the interface of most commercial APIs and the basis of existing validation techniques: retrieval-based verification, secondary LLM judges, symbolic validators, and application-specific consistency checks. With no internal telemetry, verification

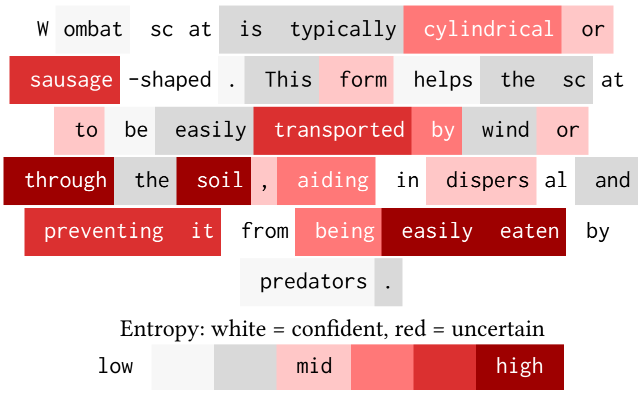

Figure 2. Generated output along with its entropy trace for the query *What shape is wombat scat?* The generated answer is a fabrication—wombat scat is distinctively cube-shaped.

must rely entirely on properties of the output, often at the cost of expensive external computation.

Tier 1: Confidence Observability. Tier 1 augments the generated text with lightweight telemetry produced during decoding. Examples include token probabilities, entropy [9, 19], confidence scores, and other decoding statistics extractable with negligible overhead. These give a coarse estimate of the model's uncertainty [16] and identify portions of the output that may warrant validation, but they are often imperfectly calibrated and may fail to distinguish confidently stated hallucinations from grounded generations.

Tier 2: Representation Observability. Tier 2 exposes information derived from the model's internal representations. Systems may access hidden-state embeddings, layer activations, attention statistics, or learned probes [1, 2] that estimate properties of the latent epistemic state. Simple probes can catch deviant model behaviors [4, 5, 11], and semantic entropy probes [7, 8] and hidden-state classifiers [21] show that these representations carry substantially richer information about reliability than logits alone, at a cost still well below exhaustive external validation.

Tier 3: Mechanistic Observability. Tier 3 exposes telemetry obtained through mechanistic analysis of the model's computation. Mechanistic techniques trace information flow through neurons, attention heads, and latent features [17]. We do not claim that Tier 3 identifies the *cause* of a hallucination in the interventional sense (a component which, when altered, changes the outcome); that is the aim of mechanistic interpretability, whose tools Tier 3 exposes but does not itself deliver. The claim is narrower: Tier 3 telemetry can distinguish between failure modes—a response with no stable internal support versus one produced by a confident but spurious pathway—rather than only flagging that a failure occurred.

3.2 Tier 1 Tensor Interface

We propose a minimal tensor interface for low-cost observability, exporting Tier 1 decoding telemetry (the per-token entropy trace) with one lightweight Tier 2 signal (attention summary statistics). We claim the tensor reveals whether the model is operating in *grounded mode* (retrieving from stable representations) or *fabrication mode* (generating without anchoring). The interface returns: (i) **Text**: The linearized response string. (ii) **Entropy trace**: Per-token entropy of the output distribution during generation. (iii) **Attention summary**: Statistics of attention patterns (concentration, self-attention ratio) from deeper processing layers.

Implementation. The tensor interface requires no architectural changes: generation APIs already return logits, from which the entropy trace is a simple transformation, and attention patterns are available with `output_attentions=True` in standard frameworks. We provide a reference implementation[1] supporting OLMo models that demonstrates the interface is practical. Overhead is modest: 2–7% added latency relative to generation, depending on signal set.

Threat model. Telemetry is produced by the same model that produces the testimony, so it is not fixed: fine-tuning moves it, and a model optimized against a monitored signal could learn to move it deliberately. Our claim is therefore not that telemetry is unfakeable, but that it is not *separately* controllable. Behavioral signals (response length, hedging) can be adapted by training without changing what the model computes; the entropy trace and attention geometry cannot change without the computation that produced the text changing too, and under current objectives nothing rewards that change. Independent work on persona monitoring supports the asymmetry: activation projections reveal distinct internal configurations for different epistemic conditions even when the text is similar [10]. Two limits follow. First, the property holds under *current* pipelines: the models we evaluate are instruction-tuned, so standard post-training did not erase the signal in our data, but optimization that includes the monitor in the objective is a different regime about which we make no claim, and the interface's long-term validity depends on it. Second, mechanistic tools can already locate the subspaces such an attack would target [18]; exporting telemetry makes manipulation auditable rather than invisible—an argument for the interface, not a guarantee of the signal.

What the tensor provides. The entropy trace provides a falsifiability signal. If a model claims certainty while its entropy trace shows high uncertainty, the mismatch is detectable. The attention summary provides a complementary signal: fabrications tend to show more dispersed attention patterns than grounded responses.

What the tensor does not provide. The tensor interface is not a lie detector. It does not prove that low-entropy outputs are true or that high-entropy outputs are false. Output

[1] https://github.com/fsgeek/pacmi26-observability, archived at DOI 10.5281/zenodo.21422488; curated from the full research record at DOI 10.5281/zenodo.21314137.

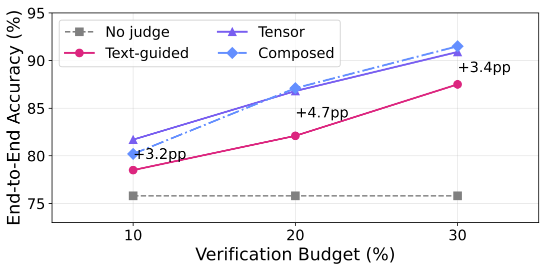

Figure 3. End-to-end accuracy vs. verification budget for four judge conditions, averaged across four architectures.

entropy also conflates epistemic uncertainty with aleatoric spread (many valid continuations); a triage signal tolerates this conflation because it ranks rather than classifies, but a classifier built on it would not. It provides *one additional signal* that, combined with other verification methods, can improve epistemic assessment.

Runtime integration. The tensor is a triage primitive, not a verdict: it ranks a model's outputs (or spans within one output) by fabrication risk, and the runtime binds rank to action under its verification budget. The bindings form a ladder ordered by cost: *abstain or flag* (withhold or annotate the output); *re-generate* (sample again or add reasoning steps and require agreement); *tool verification* (route the output to a bounded external check—a citation lookup, a database query, a test); *escalation* (request Tier 2 telemetry or a stronger fallback model for what Tier 1 cannot resolve); and *human review*, reserved for the top of the ranking. Our evaluation (§4) instantiates the first and third rungs: flagged outputs are verified and confirmed fabrications replaced by abstention, and the composed judge routes citation queries to a lookup tool because entropy inverts there. In multi-step agents the natural hook is the composition boundary, where the runtime already intercepts tool and sub-agent calls and can gate whether a flagged output propagates into the next call. We have not evaluated the interface in an agent loop, and thresholds must be calibrated on domain-specific held-out data before deployment.

Example Entropy Trace. Figure 2 shows the generated output for the query *What shape is wombat scat?* with per-token entropy color coded from white (confident) to deep red (uncertain). The trace shows where generation proceeded with high confidence (suggesting retrieval of a stored pattern) and where it proceeded with high uncertainty (suggesting fabrication).

4 Evaluation

We evaluate the return on verification investment across four strategies operating at three budget levels (the fraction of outputs the judge may verify and intervene on).

Experimental Setup. We construct a balanced set of 200 queries: 100 knowable (verifiable facts spanning multiple subjects) and 100 unknowable (fabrication prompts for nonexistent people, fictional syndromes, future events, and nonexistent citations). Ground truth labels are established by construction. We run all queries on four model families: OLMo-3 7B [14], Llama-3.1 8B [13], Qwen3 4B [15], and Mistral 7B [6], all instruction-tuned for consistent question-answering behavior.

Conditions. We evaluate each model under four conditions sharing a fixed verification budget: (i) **No judge.** Raw model output; the unverified baseline. (ii) **Text-guided judge (length).** Selects outputs for verification using response word count, a text-channel signal available without any tensor interface. (iii) **Tensor-guided judge.** Selects outputs using per-token entropy and attention summary statistics. (iv) **Composed judge.** Tensor-guided selection for general queries; bounded lookup judge for citation queries where tensor signals are known to invert. We evaluate each condition at 10%, 20%, and 30% verification budgets and measure end-to-end accuracy.

Figure 3 shows that **tensor-guided verification outperforms the text baseline at every budget level**: +3.2pp at 10%, +4.7pp at 20%, +3.4pp at 30%. Each line maps an investment strategy to its return: per-token entropy discriminates among outputs of similar length, reaching cases the text channel cannot. Per-model results confirm the pattern is consistent across all four tested model families. At 10% budget, tensor-guided verification improves accuracy over the unverified baseline for every model tested: OLMo (69.5% → 75.1%), Llama (82.5% → 88.1%), Qwen (87.0% → 91.7%), and Mistral (64.0% → 72.0%). The signal is not confined to small local models: using only the top-5 log-probabilities that serverless APIs return (a lower bound on full-vocabulary entropy), mean entropy separates knowable from unknowable queries on five further architectures from 4B to 235B parameters, including two mixture-of-experts models, with AUC 0.65–0.88 [12].

Composed judges and complementary failure modes. At 30% budget, the composed judge (91.5%) outperforms the tensor-guided judge alone (90.9%). The advantage is small in aggregate but structurally significant: it comes entirely from citation queries where entropy signals *invert*. Fabricated citations produce lower entropy than real ones because the model generates plausible fakes more fluently than it retrieves specific bibliographic details.

5 Conclusion

Systems built on language model components must decide how much verification to buy and where to spend it. We introduce epistemic observability—a hierarchy of tiers, each exposing richer epistemic information at greater cost—to provide the cost surface for that decision, and build a low-tier tensor interface that reduces the verification budget needed to curb hallucinations.

References

[1] Guillaume Alain and Yoshua Bengio. 2016. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644* (2016).

[2] Yonatan Belinkov. 2022. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics* 48, 1 (2022), 207–219.

[3] Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. 2023. Discovering Latent Knowledge in Language Models Without Supervision. In *International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=ETKGuby0hcs

[4] Nicholas Goldowsky-Dill, Bilal Chughtai, Stefan Heimersheim, and Marius Hobbhahn. 2025. Detecting strategic deception using linear probes. *arXiv preprint arXiv:2502.03407* (2025).

[5] Jiatong Han, Neil Band, Muhammed Razzak, Jannik Kossen, Tim GJ Rudner, and Yarin Gal. 2025. Simple factuality probes detect hallucinations in long-form natural language generation. *Findings of the Association for Computational Linguistics: EMNLP* (2025), 16209–16226.

[6] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. arXiv:2310.06825 [cs.CL] https://arxiv.org/abs/2310.06825

[7] Jannik Kossen, Jiatong Han, Muhammed Razzak, Lisa Schut, Shreshth Malik, and Yarin Gal. 2024. Semantic entropy probes: Robust and cheap hallucination detection in llms. *arXiv preprint arXiv:2406.15927* (2024).

[8] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. 2023. Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation. arXiv:2302.09664 [cs.CL] https://arxiv.org/abs/2302.09664

[9] Chen Ling, Xujiang Zhao, Xuchao Zhang, Wei Cheng, Yanchi Liu, Yiyou Sun, Mika Oishi, Takao Osaki, Katsushi Matsuda, Jie Ji, et al. 2024. Uncertainty Quantification for In-Context Learning of Large Language Models. *arXiv preprint arXiv:2402.10189* (2024).

[10] Christina Lu, Jack Gallagher, Jonathan Michala, Kyle Fish, and Jack Lindsey. 2026. The Assistant Axis: Situating and Stabilizing the Default Persona of Language Models. arXiv:2601.10387 [cs.CL] https://arxiv.org/abs/2601.10387

[11] Monte MacDiarmid, Timothy Maxwell, Nicholas Schiefer, Jesse Mu, Jared Kaplan, David Duvenaud, Sam Bowman, Alex Tamkin, Ethan Perez, Mrinank Sharma, Carson Denison, and Evan Hubinger. 2024. *Simple probes can catch sleeper agents*. https://www.anthropic.com/news/probes-catch-sleeper-agents

[12] Tony Mason and Vaastav Anand. 2026. Epistemic Observability in Language Models. arXiv:2603.20531 https://arxiv.org/abs/2603.20531

[13] Meta AI. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] https://arxiv.org/abs/2407.21783

[14] Team Olmo, :, Allyson Ettinger, Amanda Bertsch, Bailey Kuehl, David Graham, David Heineman, Dirk Groeneveld, Faeze Brahman, Finbarr Timbers, Hamish Ivison, Jacob Morrison, Jake Poznanski, Kyle Lo, Luca Soldaini, Matt Jordan, Mayee Chen, Michael Noukhovitch, Nathan Lambert, Pete Walsh, Pradeep Dasigi, Robert Berry, Saumya Malik, Saurabh Shah, Scott Geng, Shane Arora, Shashank Gupta, Taira Anderson, Teng Xiao, Tyler Murray, Tyler Romero, Victoria Graf, Akari Asai, Akshita Bhagia, Alexander Wettig, Alisa Liu, Aman Rangapur, Chloe Anastasiades, Costa Huang, Dustin Schwenk, Harsh Trivedi, Ian Magnusson, Jaron Lochner, Jiacheng Liu, Lester James V. Miranda, Maarten Sap, Malia Morgan, Michael Schmitz, Michal Guerquin, Michael Wilson, Regan Huff, Ronan Le Bras, Rui Xin, Rulin Shao, Sam Skjonsberg, Shannon Zejiang Shen, Shuyue Stella Li, Tucker Wilde, Valentina Pyatkin, Will Merrill, Yapei Chang, Yuling Gu, Zhiyuan Zeng, Ashish Sabharwal, Luke Zettlemoyer, Pang Wei Koh, Ali Farhadi, Noah A. Smith, and Hannaneh Hajishirzi. 2025. Olmo 3. arXiv:2512.13961 [cs.CL] https://arxiv.org/abs/2512.13961

[15] Qwen Team. 2025. Qwen3 Technical Report. Alibaba Cloud technical report. https://qwenlm.github.io/blog/qwen3/

[16] Ola Shorinwa, Zhiting Mei, Justin Lidard, Allen Z Ren, and Anirudha Majumdar. 2025. A survey on uncertainty quantification of large language models: Taxonomy, open research challenges, and future directions. *Comput. Surveys* 58, 3 (2025), 1–38.

[17] Shriyank Somvanshi, Md Monzurul Islam, Amir Rafe, Anannya Ghosh Tusti, Arka Chakraborty, Anika Baitullah, Tausif Islam Chowdhury, Nawaf Alnawmasi, Anandi Dutta, and Subasish Das. 2026. Bridging the black box: a survey on mechanistic interpretability in AI. *Comput. Surveys* 58, 8 (2026), 1–35.

[18] Thomas Winninger, Boussad Addad, and Katarzyna Kapusta. 2025. Using Mechanistic Interpretability to Craft Adversarial Attacks against Large Language Models. arXiv:2503.06269 [cs.LG] https://arxiv.org/abs/2503.06269

[19] Yijun Xiao and William Yang Wang. 2021. On hallucination and predictive uncertainty in conditional language generation. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*. 2734–2744.

[20] Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. 2024. Hallucination is Inevitable: An Innate Limitation of Large Language Models. arXiv:2401.11817 [cs.CL]

[21] Zhenliang Zhang, Xinyu Hu, Huixuan Zhang, Junzhe Zhang, and Xiaojun Wan. 2025. ICR probe: Tracking hidden state dynamics for reliable hallucination detection in LLMs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 17986–18002.